Chapter 2: Review of Object Orientation

Definitions

Object-oriented foundations

- **Abstraction:** Emphasizing relevant characteristics while leaving out unnecessary detail.
- **Procedural abstraction:** Representing a computation as a named procedure with a defined purpose and interface.
- **Data abstraction:** Representing information about an entity through a defined data model and operations while hiding unnecessary representation details.
- **Object orientation:** Organizing software as collaborating objects that combine data with the behaviour operating on it.
- **Modularity:** Dividing software into manageable units with clear responsibilities and interfaces.
- **Encapsulation:** Grouping state and behaviour within a unit such as a class and controlling access to it.
- **Information hiding:** Concealing implementation decisions behind an interface so other code need not depend on those details.
- **Object / instance:** An individual runtime entity with identity, state, and behaviour, created according to a class.
- **Class:** A description of the common structure and behaviour of a set of objects.
- **Identity:** What distinguishes an object from other objects, even when their data values are equal.
- **State:** The values and relationships held by an object at a particular time.
- **Behaviour:** The actions an object can perform and how it responds to operations.
- **Instance variable:** A field holding information separately for each instance of a class.
- **Attribute:** A property describing an object, commonly represented by a simple data value.
- **Association:** A relationship between objects.
- **Reference variable:** A variable whose value refers to an object; the variable is not the object itself.
- **Declared type:** The type specified for a variable, which determines the operations available through that variable at compile time.
- **Runtime class:** The actual class of an object during execution.

- **Class / static variable:** A field belonging to a class rather than separately to each instance.
- **Constant:** A named value intended to remain unchanged. A final reference cannot be reassigned, but its referenced object may still be mutable.
- **Mutable state:** Information that can change during execution.

Methods, inheritance, and polymorphism

- **Operation:** A description of behaviour independently of a particular implementation.
- **Method:** A class's implementation of an operation.
- **Polymorphism:** The ability to invoke the same operation on different kinds of objects and obtain the implementation appropriate to each object.
- **Inheritance:** A mechanism through which a subclass receives applicable features from a superclass and can add or specialize behaviour.
- **Superclass:** A more general class from which another class inherits.
- **Subclass:** A more specialized class that inherits from another class.
- **Inheritance hierarchy:** The organization of classes through superclass-subclass relationships.
- **Is-a rule:** The requirement that a subclass genuinely represents a kind of its superclass and supports the superclass contract.
- **Part-of relationship:** A relationship in which one object forms part of another, represented through associations rather than inheritance alone.
- **Contract:** The promised behaviour and conditions that users of a class or operation may rely on.
- **Abstract method:** A method declared without an implementation that concrete subclasses must implement or inherit an implementation for.
- **Abstract class:** A class that cannot be instantiated directly and may leave some operations unimplemented.
- **Concrete class:** A class that can be instantiated and provides implementations for all required abstract methods.
- **Overriding:** Supplying a subclass implementation of an inherited instance method.
- **Overloading:** Defining methods with the same name but different parameter lists.
- **Dynamic binding:** Selecting an overridden instance method at runtime according to the actual class of the receiving object.

- **Cast:** An explicit conversion of the type through which a value is viewed. Casting a reference does not change the object's class.
- **Downcast:** Casting a reference to a more specific type; an incompatible downcast fails at runtime.

Java language and collections

- **Bytecode:** Instructions produced by the Java compiler for execution by a Java virtual machine.
- **Java virtual machine (JVM):** The execution environment that runs Java bytecode.
- **API:** The accessible types and operations through which code uses a software component.
- **Javadoc:** A tool and documentation-comment format for generating API documentation from Java source code.
- `char`**:** A Java primitive value representing one UTF-16 code unit.
- **Unicode:** A standard assigning code points to characters used in writing systems and symbols.
- **UTF-16 code unit:** A 16-bit unit used to encode Unicode text; some characters require two such units.
- **Wrapper class:** An object type representing a primitive value, such as `Character` for `char`.
- `String`**:** Java's immutable representation of a sequence of text characters.
- **Concatenation:** Joining text sequences together, for example using `+` with strings.
- **Array:** An indexed container with a fixed length and a declared element type.
- **Collection:** An object that groups elements; Java's `Collection` interface defines common collection operations.
- `ArrayList`**:** A resizable, indexed list implementation in Java.
- **Iterator:** An object used to traverse elements, commonly through `hasNext()` and `next()`.
- **Generics:** Type parameters that express the types a class or method works with, improving compile-time type checking.
- **Exception:** An object signalling a failure or abnormal condition that interrupts normal execution flow.
- `try/catch`**:** Java constructs that run code and handle exceptions of specified types.
- **Interface:** A contract that implementing classes promise to support; variables can use an interface type to refer to different implementations.

- **Default interface method:** An instance method with an implementation supplied by an interface.
- **Static method:** A method associated with a class or interface rather than invoked through instance-method dynamic binding.
- `Comparable`**:** An interface for defining an object's natural ordering through comparison with another object.

Organization, access, and execution

- **Package:** A grouping of related Java types that provides a namespace and an access boundary.
- **Namespace:** A naming context that distinguishes otherwise identical names.
- **Import:** A declaration allowing a type or static member to be referenced by a shorter name in source code.
- **Visibility / access control:** Rules determining which code may use a type or member.
- `public`**:** An access modifier permitting broad access, subject to the accessibility of the enclosing type and module.
- `private`**:** An access modifier limiting access to the declaring class and its enclosing nest of related nested classes.
- **Package access:** Access available within the same package when no access modifier is specified.
- `protected`**:** Access within the package and from subclasses, with additional restrictions for subclass access across packages.
- **Thread:** A sequence of execution that can progress concurrently with other threads.
- **Concurrency:** Multiple activities making progress during overlapping periods, whether or not they execute simultaneously.
- `Runnable`**:** An interface defining a `run()` operation for a task that can be executed by a thread.
- `Thread.start()`**:** An operation that starts a separate thread which executes its task's `run()` method.
- `run()`**:** The method containing a thread task's work; calling it directly executes on the calling thread.
- **User interface (UI):** The part of a system through which users provide input and receive output.
- **Domain logic:** Code implementing the rules and operations of the problem domain.
- **Deprecated feature:** A feature marked as discouraged for continued use, often because a preferred replacement exists.

2.0 What is Object Orientation?

- Procedural abstraction organizes computation into procedures. Data abstraction groups information describing an entity.
- Object orientation places behaviour with the data it operates on. A running program consists of collaborating objects.
- Abstraction emphasizes relevant characteristics. Modularity divides code into manageable units. Encapsulation groups state and behaviour and supports information hiding.

2.1 Classes and Objects

- An object is an individual runtime entity with identity, state, and behaviour. A class describes its instances' common structure and behaviour.
- Film can be a class and a particular film an instance. Genre can be a class and Science Fiction one instance.
- Choose singular, descriptive class names such as BankAccount, with the right level of generality and one clear meaning.

2.2 Instance Variables

- Instance variables hold information for each object. Attributes hold simple values; associations represent relationships with other objects.
- A reference variable refers to an object; it is not the object itself. Multiple variables can refer to the same object, and a variable may later refer to another object.
- The variable's declared type determines which operations are available through it.

2.2.0 Class variables

- A static variable belongs to the class and is shared among its instances.
- Constants and lookup data are common uses. Mutable shared state needs care because a change affects all its users.

2.3 Methods, Operations, and Polymorphism

- An operation describes behaviour independently of its implementation. A method supplies a class's implementation.
- Polymorphism allows the same operation to behave differently for different kinds of objects; circles and rectangles calculate area differently.
- Polymorphic methods reduce repeated type tests and switch statements.

2.4 Inheritance Hierarchies

- A superclass defines common features. Subclasses inherit applicable features and may add or specialize behaviour.
- Apply the is-a rule: a checking account is an account, but a province is not a country.
- Every inherited feature must make sense in the subclass. A part-of relationship is an association rather than inheritance.

2.5 Inheritance, Polymorphism, and Variables

2.5.0 Abstract classes and overriding

- Declare an operation at the highest sensible level. An abstract method specifies behaviour without implementing it there.
- An abstract class cannot be instantiated. A concrete class must implement or inherit implementations of all required abstract methods.
- Overriding supplies a subclass implementation of an inherited instance method. It can extend or optimize behaviour while preserving the contract.
- Overloading uses different parameter lists and is distinct from overriding.

2.5.1 Dynamic binding

- The declared type controls whether a call is legal; the runtime object's class determines which overridden instance method executes.
- Look first for an implementation in the runtime class, then up its superclass chain.
- Example: Shape s = new Circle(); s.area(); executes the Circle implementation.
- Casting changes the type through which a reference is viewed, not the object's class. An invalid downcast can fail at runtime.

2.6 Java Fundamentals

2.6.0 Language and documentation

- The lecture traces object orientation through Simula, Smalltalk, and C++, then Java's development from Oak and its 1995 launch.
- Java compiles to bytecode executed by a virtual machine. Read class and method documentation to understand library behaviour.
- Javadoc generates API documentation from source declarations and comments.

2.6.1 Characters, strings, and collections

- char holds one UTF-16 code unit; Character is its wrapper class. Some Unicode characters require two code units.
- String represents text; + concatenates strings.
- Arrays have fixed length. ArrayList supports a resizable collection. Iterators traverse elements using hasNext() and next().
- Generics express element types: ArrayList<String> names = new ArrayList<>(); avoids many explicit casts during retrieval.

2.6.2 Exceptions and interfaces

- Exceptions signal failures. A try/catch block handles specified types of exceptions; the handling should reflect the actual problem.
- An interface specifies behaviour implementing classes promise to provide. A class can implement multiple interfaces, and a variable can have an interface type.
- Runnable, Collection, Iterator, and Comparable are library examples.
- The slides' prohibition on implementation code in interfaces describes older Java. Modern Java permits default and static interface methods.

2.6.3 Packages and access

- Packages group related classes and provide namespaces. Imports allow short type names in source code.
- public permits broad access; private hides implementation within the class and its enclosing nest; no modifier gives package access.
- protected permits package access and subclass access, subject to Java's additional rules across packages.
- Use the narrowest practical visibility.

2.6.4 Threads

- A thread is an execution sequence that can progress concurrently with other threads.
- Implement Runnable.run(), create a Thread using that Runnable, and call start(). Alternatively, subclass Thread.
- Calling run() directly uses the calling thread; start() begins a separate thread of execution.
- Coordinate access when threads share mutable state.

2.7 Programming Style

- Prefer understandable code and descriptive names. Shorter or cleverer code is not automatically better.
- Explain non-obvious decisions in comments and organize class elements consistently.
- Extract shared behaviour into methods instead of copying code, so fixes do not need to be repeated.
- Follow the is-a rule, hide implementation details, and separate UI code from domain logic to improve reuse.

2.8 Difficulties and Risks

- Languages and libraries evolve. Deprecated features may need replacement during maintenance.
- Measure efficiency against the application's requirements. Virtual-machine execution and dynamic binding alone do not establish that a program is too slow.